

소켓프로그래밍 4

김학재

2026.08.25

목차

1. select
2. epoll
3. io_uring
4. 채팅 프로그램

지난시간 + 부가설명

한 서버가 여러 클라이언트의 통신을 동시에 처리하기 위해서

1) 멀티프로세싱 – fork 방식

- 프로세스 생성에 많은 시간과 자원필요
- 프로세스간의 통신 복잡

2) 멀티스레딩 – thread 방식

- 스레드의 생성 및 컨텍스트 스위칭이 프로세스보다는 빠르고 가볍다.
- 스레드간의 통신은 덜 복잡 (메모리 공유해서)

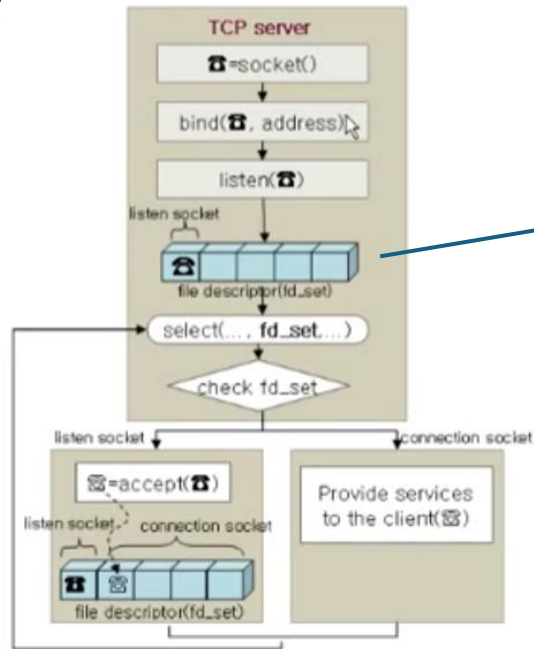
3) 멀티플렉싱 – select() 방식

- 싱글 프로세스, 싱글 스레드로 여러 클라이언트와 통신 가능 (컨텍스트 스위칭 X)
- 멀티플렉싱은 엄밀히 말하면 여러 클라이언트와의 통신을 동시에 실행하지는 않는다는 점. (멀티코어인 경우)
- 개념적으로 병렬성은 없고, 동시성만 있다.

select 방식

```
int select (  
    int                nfd,  
    fd_set             *readfds,  
    fd_set             *writefds,  
    fd_set             *exceptfds,  
    struct timeval     *timeout  
);
```

: 조건을 만족한 fd 총 개수 반환, readfds수정
: 검사할 소켓(파일) 범위
: 읽을 수 있는 fd 비트맵
: 쓸 수 있는 fd 비트맵
: 예외 상황 있는지 fd 비트맵
: select함수 호출시 커널에서 머무를 최대시간



fd (소켓)
비트맵

0	1	2	3	4	5
0	0	0	1	0	1

do_select()

usr > src > linux-source-6.8.0 > fs >  select.c

```
static ninline_for_stack int do_select(int n, fd_set_bits *fds, struct timespec64 *end_time)
```

```
{
```

```
477 {  
478     ktime_t expire, *to = NULL;  
479     struct poll_wqueues table;  
480     poll_table *wait;  
481     int retval, i, timed_out = 0;  
482     u64 slack = 0;  
483     __poll_t busy_flag = net_busy_loop_on() ? POLL_BUSY_LOOP : 0;  
484     unsigned long busy_start = 0;  
485  
486     rcu_read_lock();  
487     retval = max_select_fd(n, fds);  
488     rcu_read_unlock();  
489  
490     if (retval < 0)  
491         return retval;  
492     n = retval;  
493  
494     poll_initwait(&table);  
495     wait = &table.pt;  
496     if (end_time && !end_time->tv_sec && !end_time->tv_nsec) {  
497         wait->qproc = NULL;  
498         timed_out = 1;  
499     }
```

n : 감시할 fd의 개수(범위)

fds : 감시할 파일 디스크립터 집합

end_time : select가 얼마나 기다릴지 나타내는 시간정보

fd가 1000번까지 있다고 칠 때 만약

fd 59, 125, 127번이 감시가 필요하다면 127번까지만 순회하도록
최대 어디까지 검사할 것인지 정해줌

RCU : 리눅스 커널의 동기화

do_select()

```
505 for (;;) {
506     unsigned long *rinp, *routp, *rexp, *inp, *outp, *exp;
507     bool can_busy_loop = false;
508
509     inp = fds->in; outp = fds->out; exp = fds->ex;
510     rinp = fds->res_in; routp = fds->res_out; rexp = fds->res_ex;
511
512     for (i = 0; i < n; ++rinp, ++routp, ++rexp) {
513         unsigned long in, out, ex, all_bits, bit = 1, j;
514         unsigned long res_in = 0, res_out = 0, res_ex = 0;
515         __poll_t mask;
516
517         in = *inp++; out = *outp++; ex = *exp++;
518         all_bits = in | out | ex;
519         if (all_bits == 0) {
520             i += BITS_PER_LONG;
521             continue;
522         }
523
524         for (j = 0; j < BITS_PER_LONG; ++j, ++i, bit <= 1) {
525             struct fd f;
526             if (i >= n)
527                 break;
528             if (!(bit & all_bits))
529                 continue;
```

bit = 00000001

fd

in 비트맵

out 비트맵

ex 비트맵

0	1	2	3	4	5	6	7
0	0	0	1	0	1	0	0
0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0

0	0	0	1	0	1	0	1
---	---	---	---	---	---	---	---

all bits = 10101000

bit = 00000001

bit & all_bits = 00000000

별일 없는 fd는 continue

```

524 for (j = 0; j < BITS_PER_LONG; ++j, ++i, bit <= 1) {
525     struct fd f;
526     if (i >= n) → 범위
527         break;
528     if (!(bit & all_bits))
529         continue;
530     mask = EPOLLNVAL;
531     f = fdget(i);
532     if (f.file) {
533         wait_key_set(wait, in, out, bit,
534                     busy_flag);
535         mask = vfs_poll(f.file, wait);
536
537         fdput(f);
538     }
539     if ((mask & POLLIN_SET) && (in & bit)) {
540         res_in |= bit;
541         retval++;
542         wait->_qproc = NULL;
543     }
544     if ((mask & POLLOUT_SET) && (out & bit)) {
545         res_out |= bit;
546         retval++;
547         wait->_qproc = NULL;
548     }
549     if ((mask & POLLEX_SET) && (ex & bit)) {
550         res_ex |= bit;
551         retval++;
552         wait->_qproc = NULL;
553     }
554     /* got something, stop busy polling */
555     if (retval) {
556         can_busy_loop = false;
557         busy_flag = 0;

```

fdget : fd i번이 가리키는 파일/소켓 객체가 뭔지

f.file : fd i가 가리키는 커널 내부의 struct file 객체

if(f.file) 실제로 fd가 유효한지 확인

wait_key_set(...) 이 fd에서 어떤 이벤트를 기다리고 있는지

vfs_poll : 지금 I/O가 가능한지 확인, 불가능하면 wait queue에 등록

결국 :

select는 준비된 fd 총 개수를 반환하고

fd_set을 업데이트 한다.

select() 흐름 :

fd들 검사 O(n)

-> 준비된 fd 있으면 바로 반환 / 없으면 wait queue에 등록

-> sleep

-> 어떤 fd에서 이벤트 발생 -> wakeup -> select 실행

-> 다시 fd 검사

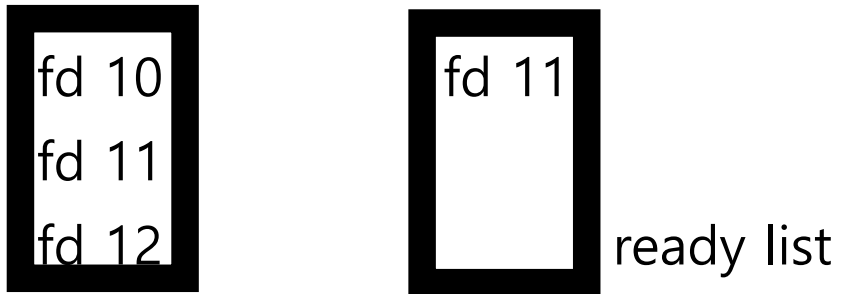
epoll

select는 이벤트가 발생할 때마다 fd들을 매번 처음부터 검사한다.

epoll은 이벤트가 발생한 fd만 따로 모아둔다.

epoll_ctl() : 무슨 fd를 감시할지 등록 및 관리

epoll_wait() : 감시 중인 fd 중 이벤트 발생한 게 있는지



epoll_ctl 에서 감시하는 fd에서 관심 이벤트가 발생하면 ready lis로 들어감

```
epoll_ctl(epfd, EPOLL_CTL_ADD, clientfd, &event);
```

그 후

epoll_wait()에서 대기하다가 이벤트 발생 시 ready list 반환

```

2639     int do_epoll_ctl_file(struct file *f, int op, struct epoll_key *tf,
2640                          struct epoll_event *epds, bool nonblock)
2641     {
2688         epi = ep_find(ep, tf);
2689
2690         error = -EINVAL;
2691         switch (op) {
2692         case EPOLL_CTL_ADD:
2693             if (!epi) {
2694                 epds->events |= EPOLLERR | EPOLLHUP;
2695                 error = ep_insert(&ctx, ep, epds, tf, full_check);
2696             } else
2697                 error = -EEXIST;
2698             break;
2699         case EPOLL_CTL_DEL:
2700             if (epi) {
2701                 /*
2702                  * The eventpoll itself is still alive: the refcount
2703                  * can't go to zero here.
2704                  */
2705                 ep_remove(ep, epi);
2706                 error = 0;
2707             } else {
2708                 error = -ENOENT;
2709             }
2710             break;
2711         case EPOLL_CTL_MOD:
2712             if (epi) {
2713                 if (!(epi->event.events & EPOLLEXCLUSIVE)) {
2714                     epds->events |= EPOLLERR | EPOLLHUP;
2715                     error = ep_modify(ep, epi, epds);
2716                 }
2717             } else
2718                 error = -ENOENT;
2719             break;
2720         }
2721
2722         ep_ctl_unlock(&ctx, ep, full_check);
2723         return error;
2724     }

```

EPOLL_CTL_ADD : epfd에 clientfd를 추가
 EPOLL_CTL_DEL : 삭제
 EPOLL_CTL_MOD : 변경

io_uring

프로그램이 커널에게 I/O 작업을 제출하고 커널이 작업을 완료하면 완료 결과를 알려준다.

epoll 은 I/O준비 상태를 알려준다면
io_uring은 I/O작업 자체를 요청한다
I/O를 비동기적으로 처리할 수 있다.

시스템 콜을 줄일 수 있다.

submission queue, completion queue를 공유 메모리 기반으로 활용해서 여러 작업을 한꺼번에 제출, 및 가져오기 가능.

->커널에 한꺼번에 모아서 갔다오는 느낌 인 듯 하다. 버퍼느낌으로 쌓아서 제출 및 가져오기

비동기 Asynchronous : 기다리지 않고 다른일 진행

병렬성 Parallel : 여러작업 동시에 실행

동시성 Concurrency : 여러 작업 번갈아 진행해서 동시에 진행되는 것 처럼 처리

epoll은 I/O 자체를 해주는게 아님, 많은 I/O 작업을 처리할 때 시스템 콜 계속 발생.(read,write)

io_uring은 많은 I/O를 처리하기 좋음, 및 I/O요청하고 다른 일 가능.

단점은 : 복잡하다.

채팅 프로그램

```
k@k:/mnt/c/Users/k/OS$ ./server
```

```
서버 대기 중...
```

```
클라이언트 연결됨!
```

```
Client: 오 연결 됐다안녕!
```

```
Server: 그래 수고많았다.
```

```
|
```

```
[(base) hakbook@hakbook TCP % ./client
```

```
서버에 연결됨!
```

```
Client: 오 연결 됐다! 안녕!
```

```
Server: 그래 수고많았다.
```

```
Client: □
```

wsl <-> mac 채팅 프로그램

wsl은 windows 에서 실행되는 Linux 환경이며,
가상 네트워크를 사용한다.

window 서버 192.168.XX.XX : 5000 으로 들어오는 TCP
연결을

wsl의 172.21.XXX.XXX:5000로 넘김 (포트 포워딩)

mac에서 192.168.XX.XX 저기로 connect함

감사합니다